

Estrutura de Dados

Grafos: Conceitos e Representações

[Baixe aqui os exemplos da aula](#)



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Da Hierarquia à Malha

A árvore organiza dados numa hierarquia: cada nó tem um único pai, e não há caminho de volta a um ancestral. A malha viária do LogiRota não obedece essa regra — duas ruas podem formar um ciclo, e um ponto pode ser alcançado por mais de um caminho. Essa relação exige uma estrutura diferente: o grafo, sem raiz e sem hierarquia entre seus elementos.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Vértice, Aresta e Peso

Um grafo é definido por dois conjuntos: vértices, os elementos da estrutura, e arestas, as ligações entre pares de vértices. No LogiRota, cada Ponto de entrega cadastrado nas aulas anteriores vira um vértice, e cada rua entre dois pontos vira uma aresta. O peso da aresta é a distância que `distancia_ate` já calculava — nenhum número novo é inventado.



Prof. Everaldo Graciliano Souza Ribeiro

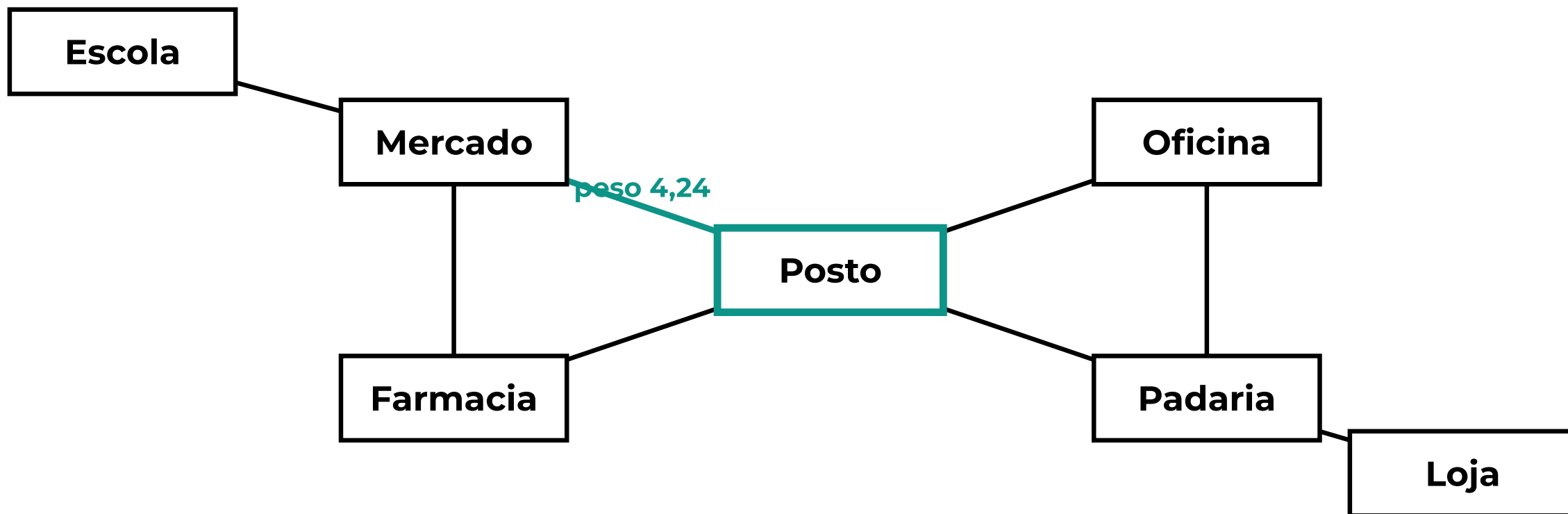
Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

A Malha Viária como Grafo



Um grafo não tem raiz: **Posto Central** alcança quatro vizinhos diretos, sem hierarquia entre eles.



Grafo Não Dirigido

Uma aresta pode ter, ou não, sentido único. Num grafo dirigido, a ligação de A para B não implica ligação de B para A — útil para ruas de mão única. O LogiRota modela a malha como grafo não dirigido: toda rua cadastrada liga os dois pontos nos dois sentidos, porque o caminhão pode percorrê-la em qualquer direção.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Duas Formas de Guardar a Malha

Definido o grafo, resta escolher como armazená-lo. Duas representações resolvem o problema de formas opostas: a matriz de adjacência reserva espaço fixo para cada par possível de vértices, enquanto a lista de adjacência guarda apenas as ligações que realmente existem. A escolha entre elas depende de quão densa a malha viária é — quantas ruas existem frente ao total possível.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Matriz de Adjacência

```
class GrafoMatriz:
    def __init__(self, pontos):
        self._pontos = list(pontos)
        self._indice = {p.nome: i for i, p in enumerate(self._pontos)}
        n = len(self._pontos)
        self._matriz = [[None] * n for _ in range(n)]

    def adicionar_rua(self, nome_a, nome_b):
        i, j = self._indice[nome_a], self._indice[nome_b]
        peso = self._pontos[i].distancia_ate(self._pontos[j])
        self._matriz[i][j] = peso
        self._matriz[j][i] = peso           # nao dirigido: os dois sentidos
```



O Custo da Matriz de Adjacência

A matriz de adjacência reserva uma célula para cada par de vértices, cadastrada a rua ou não — v vértices custam v^2 células, mesmo numa malha esparsa. Em compensação, verificar se dois pontos são vizinhos é um único acesso indexado: custo constante, igual para qualquer par, exista rua entre eles ou não.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Lista de Adjacência

```
class GrafoLista:
    def __init__(self, pontos):
        self._pontos = {p.nome: p for p in pontos}
        self._vizinhos = {nome: [] for nome in self._pontos}

    def adicionar_rua(self, nome_a, nome_b):
        peso = self._pontos[nome_a].distancia_ate(self._pontos[nome_b])
        self._vizinhos[nome_a].append((nome_b, peso))
        self._vizinhos[nome_b].append((nome_a, peso)) # nao dirigido
```



O Custo da Lista de Adjacência

A lista de adjacência guarda, para cada vértice, apenas os vizinhos que ele realmente tem — o espaço cresce com o número de ruas cadastradas, não com o quadrado dos vértices. A verificação de vizinhança paga esse espaço menor com tempo maior: é preciso percorrer a lista do vértice até encontrar o outro lado, ou concluir que não existe.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

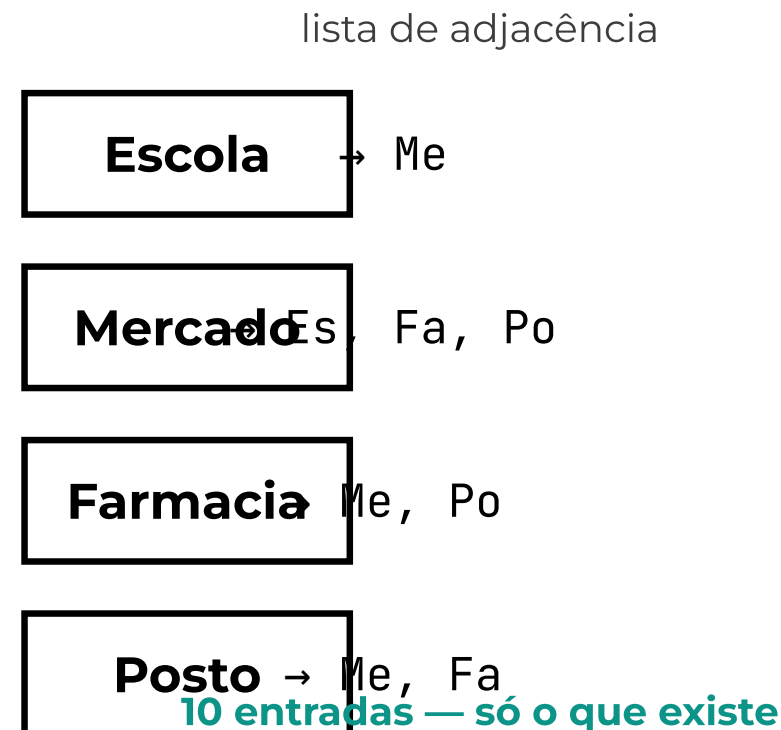
 <https://www.linkedin.com/in/everaldoribeiroads>

Matriz × Lista, Lado a Lado

matriz de adjacência

	Es	Me	Fa	Po
Es	—	.		
Me	.	—	.	.
Fa		.	—	.
Po		.	.	—

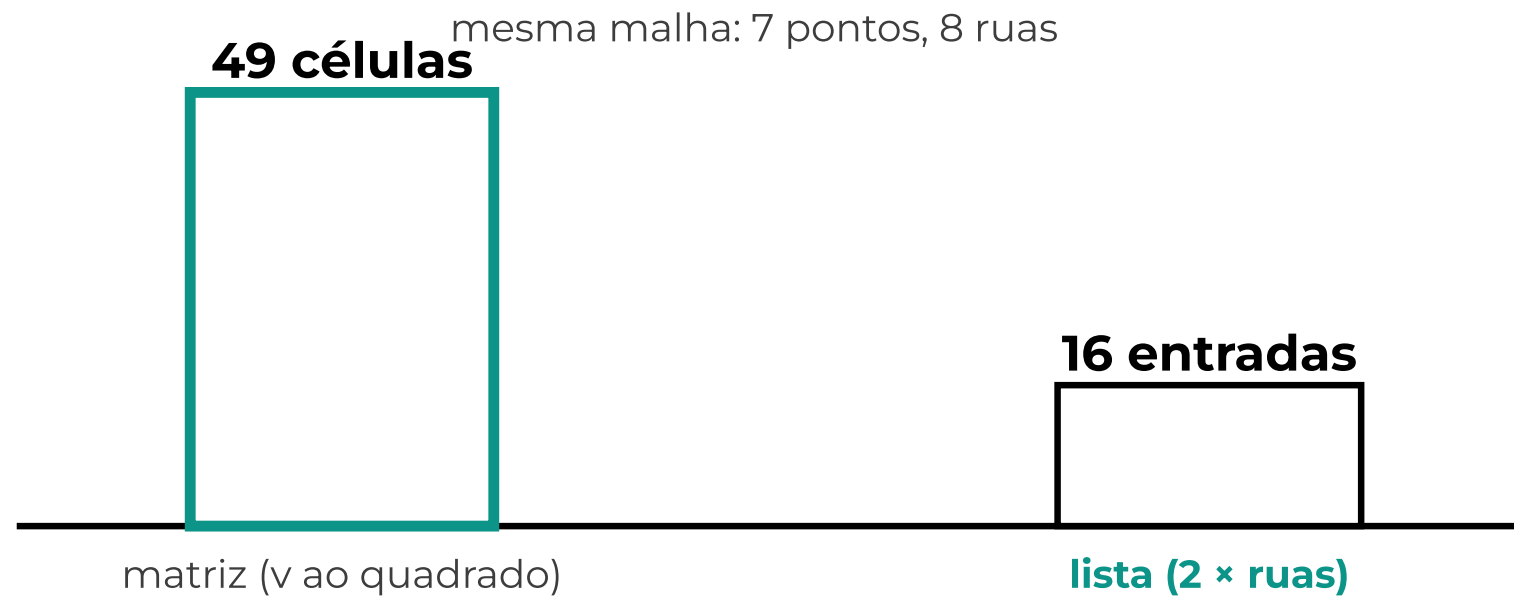
16 células, cadastrada a rua ou não



Mesmos 4 vértices, mesmas 4 ruas: **a matriz reserva espaço fixo**; a lista só guarda a ligação que existe.



Custo em Números, sobre os 7 Pontos



O espaço da matriz não depende de quantas ruas existem; **o da lista, sim.**



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Qual Representação Escolher

Não há resposta única: a matriz vale a pena quando a malha é densa, com muitas ruas frente ao total de pares possíveis, porque o espaço extra é pequeno e a vizinhança consulta em tempo constante. A lista vence quando a malha é esparsa, como costuma ser uma malha viária real — poucas ruas por ponto, frente a muitos pontos possíveis.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

As Duas Representações em Ação

```
matriz = montar(GrafoMatriz(PONTOS))
lista = montar(GrafoLista(PONTOS))

print(matriz.total_celulas())          # 49
print(lista.total_arestas_armazenadas()) # 16

a, b = "Mercado Barra", "Posto Central"
print(matriz.sao_vizinhos(a, b))       # True
print(lista.sao_vizinhos(a, b))       # True

# Mesma pergunta, mesma resposta -- custos diferentes.
```



Atividades

Exercício 1

Por que o LogiRota modela a malha viária como grafo não dirigido, e não como grafo dirigido?

Exercício 2

Com 20 pontos de entrega e 25 ruas cadastradas, quantas células a matriz reserva? Quantas entradas a lista guarda?

Exercício 3

GrafoLista.sao_vizinhos percorre uma lista em vez de indexar direto. Por que essa escolha ainda compensa numa malha esparsa?



Desafio extra

Opcional

Quem quiser ir além do combinado nesta semana:

1. Escrever `grau(nome)` nas duas representações — quantas ruas partem de um ponto — e comparar o custo de calculá-lo em cada uma.
2. Escrever `GrafoMatriz.remove_rua` e `GrafoLista.remove_rua`, desfazendo uma ligação nos dois sentidos.

Nada disso é cobrado na entrega. O projeto segue completo sem essas adições.

